

Convocatoria 1 - Proyecto 1

Alumno: Alejandro Casado Figueredo

Github: <https://github.com/alexcefe02/VIU>

0) Cargar una de las imágenes histológicas

Este código carga una imagen histológica en formato RGB desde el archivo

'histo_1.jpg', normaliza sus valores dividiendo por 255 para llevar los píxeles al rango `[0, 1]`, y la muestra con `matplotlib`.

La normalización es útil para facilitar el procesamiento posterior, ya que muchas funciones trabajan mejor con valores en coma flotante. La visualización permite verificar que la imagen se ha cargado correctamente y observar sus características iniciales.

```
In [1]: import numpy as np
import skimage.io as io
import matplotlib.pyplot as plt

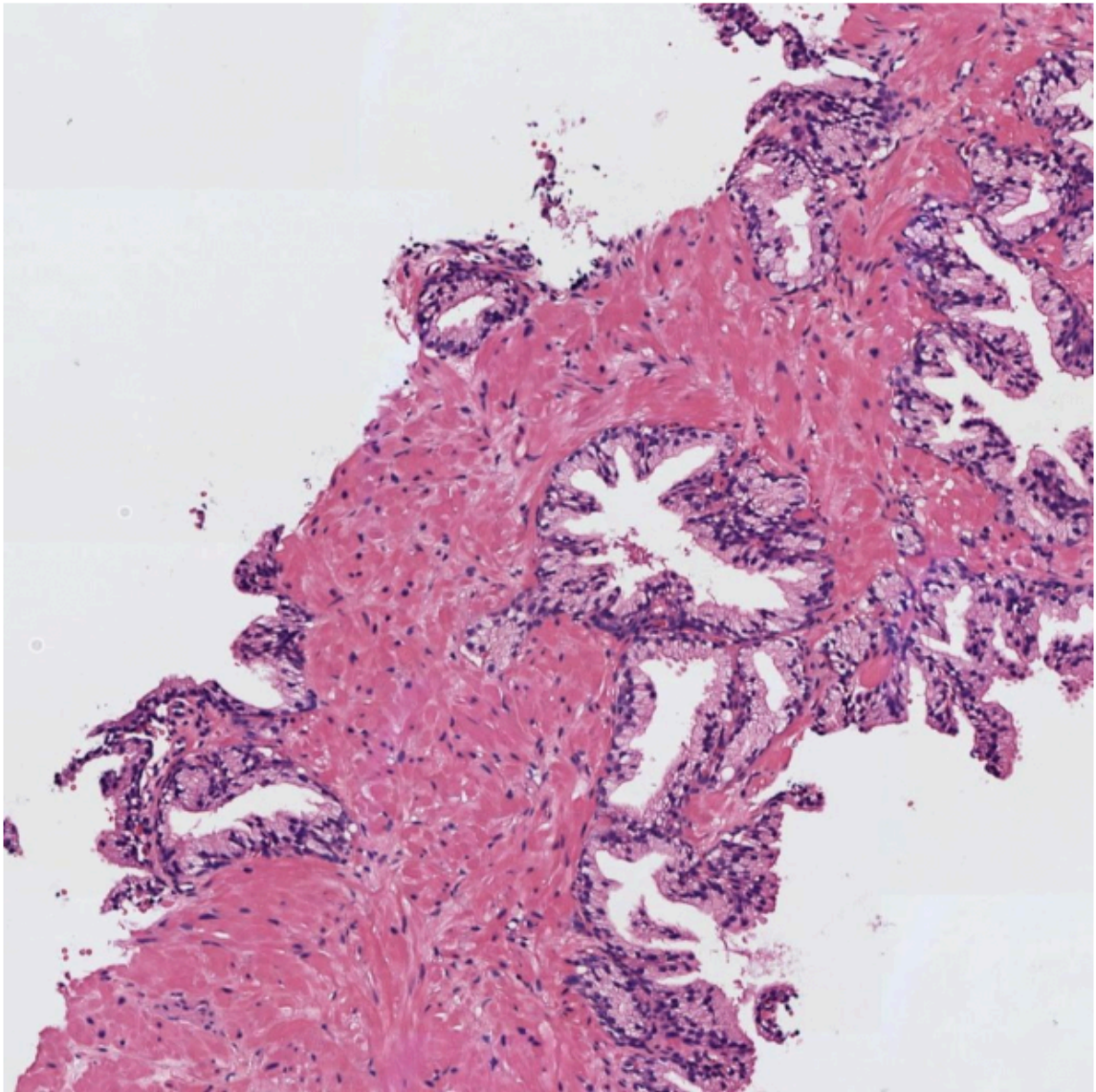
# Ruta a la imagen
image_path = 'histo_1.jpg'

# Cargar la imagen en formato RGB
image = io.imread(image_path)

# Normalizar los valores de los píxeles al rango [0, 1]
img = image / 255.0

# Visualizar la imagen
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.title('Imagen Histológica Normalizada')
plt.axis('off')
plt.show()
```

Imagen Histológica Normalizada



1) Realizar una transformación de color para convertir la imagen al espacio de color CMYK

En un primer momento, la intención ha sido convertir la imagen desde su formato RGB al espacio de color CMYK. Para ello, se realizó una transformación aproximada utilizando operaciones matriciales sobre los canales RGB, con el objetivo de obtener los componentes Cian (C), Magenta (M), Amarillo (Y) y Negro (K) por separado.

Mediante este proceso se generaron las imágenes correspondientes a cada canal del modelo CMYK, los cuales se visualizaron individualmente en escala de grises dentro de una misma figura.

```
In [2]: # Separar canales RGB
R = img[:, :, 0]
G = img[:, :, 1]
B = img[:, :, 2]

# Convertir a CMY
C = 1 - R
M = 1 - G
```

```
Y = 1 - B

# Calcular K (negro)
K = np.minimum(np.minimum(C, M), Y)

# Evitar división por cero
with np.errstate(divide='ignore', invalid='ignore'):
    C = (C - K) / (1 - K + 1e-10)
    M = (M - K) / (1 - K + 1e-10)
    Y = (Y - K) / (1 - K + 1e-10)

# Reescalar a [0,255] para visualización
C_display = (C * 255).astype(np.uint8)
M_display = (M * 255).astype(np.uint8)
Y_display = (Y * 255).astype(np.uint8)
K_display = (K * 255).astype(np.uint8)

# Visualización de los 4 canales
fig, axs = plt.subplots(2, 2, figsize=(10, 10))

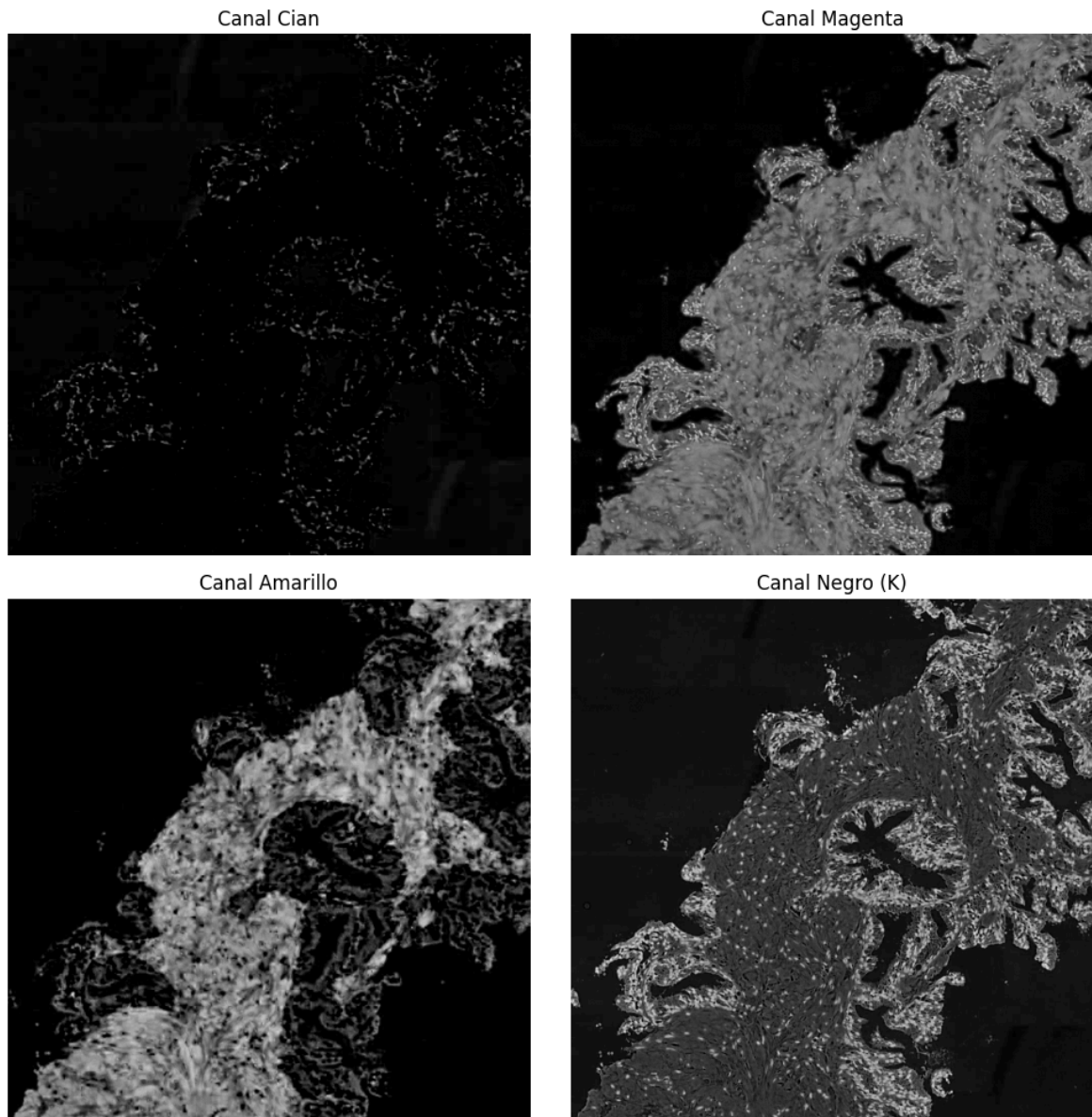
axs[0, 0].imshow(C_display, cmap='gray')
axs[0, 0].set_title('Canal Cian')
axs[0, 0].axis('off')

axs[0, 1].imshow(M_display, cmap='gray')
axs[0, 1].set_title('Canal Magenta')
axs[0, 1].axis('off')

axs[1, 0].imshow(Y_display, cmap='gray')
axs[1, 0].set_title('Canal Amarillo')
axs[1, 0].axis('off')

axs[1, 1].imshow(K_display, cmap='gray')
axs[1, 1].set_title('Canal Negro (K)')
axs[1, 1].axis('off')

plt.tight_layout()
plt.show()
```



2) Umbralizar la imagen para separar los píxeles del fondo de la región tisular

Ahora aplicaremos un filtro gaussiano al canal magenta para suavizar la imagen y reducir el ruido, lo que mejora la calidad de la segmentación. A continuación, se utiliza el método de Otsu para calcular un umbral automático que separa las regiones más claras (como el lumen y el fondo) del resto del tejido.

La salida es una máscara binaria donde los píxeles correspondientes al fondo y al lumen se marcan como `1`, y el resto como `0`. Esta operación es clave para identificar estructuras relevantes en imágenes histológicas y preparar la imagen para análisis más avanzados, como conteo o medición de áreas.

```
In [3]: from skimage.filters import gaussian, threshold_otsu
        from skimage import img_as_ubyte

        # Aplicar filtro gaussiano (5x5)
        # sigma se ajusta para un suavizado equivalente a una ventana 5x5
        blue_gauss = gaussian(M_display, sigma=1)
```

```
# Convertir a uint8 en rango [0-255] (requerido por Otsu)
blue_gauss_uint8 = img_as_ubyte(blue_gauss)

# Obtener umbral de Otsu
umbral_otsu = threshold_otsu(blue_gauss_uint8)

# Crear máscara binaria: Lumen y fondo como 1, resto como 0
mascara_binaria = blue_gauss_uint8 < umbral_otsu

# Visualizar la máscara binaria
plt.figure(figsize=(8, 8))
plt.imshow(mascara_binaria, cmap='gray')
plt.title('Máscara Binaria - Fondo y Lumen = 1')
plt.axis('off')
plt.show()
```

Máscara Binaria - Fondo y Lumen = 1



3) Limpiar la imagen eliminando los artefactos de lumen (objetos blancos pequeños que no son lúmenes)

Lo siguiente a realizar consiste en eliminar objetos pequeños de la máscara binaria resultante de la segmentación, utilizando la función `remove_small_objects`. En

concreto, descarta regiones con un área menor a 300 píxeles, consideradas como artefactos o ruido.

El resultado es una máscara más limpia y precisa, que conserva solo las regiones significativas (como lumen o espacios reales), facilitando análisis morfológicos posteriores.

```
In [4]: # Importación de la función a utilizar
from skimage.morphology import remove_small_objects

# Eliminar objetos con área menor a 300 píxeles
# La imagen debe estar en formato booleano para remove_small_objects
mascara_limpia = remove_small_objects(mascara_binaria.astype(bool), min_size=300)

# Visualizar la máscara resultante
plt.figure(figsize=(8, 8))
plt.imshow(mascara_limpia, cmap='gray')
plt.title('Máscara Limpia - Sin Artefactos Pequeños')
plt.axis('off')
plt.show()
```

Máscara Limpia - Sin Artefactos Pequeños



4) Rellenar con 0s el fondo de la imagen para quedarnos únicamente con los lúmenes

Este código elimina el fondo de la máscara binaria, conservando únicamente las regiones correspondientes a los lúmenes.

Primero, se identifican píxeles de fondo (valor 1) ubicados preferentemente en las esquinas de la imagen. Estos puntos actúan como semillas para el algoritmo de `flood_fill`, que rellena de 0 todas las regiones conectadas al fondo.

Como resultado, el fondo se elimina completamente de la máscara, dejando únicamente los lúmenes destacados, lo cual es esencial para realizar análisis cuantitativos precisos sobre estas estructuras.

```
In [5]: from skimage.segmentation import flood_fill

# --- Paso 1: función para encontrar semillas en el fondo ---
def find_seeds(image, num_seeds=1):
    seeds = []
    h, w = image.shape

    # Buscar primero en los 4 vértices
    vertices = [(0, 0), (0, w-1), (h-1, 0), (h-1, w-1)]
    for y, x in vertices:
        if image[y, x] == 1:
            seeds.append((y, x))
        if len(seeds) >= num_seeds:
            return seeds[:num_seeds]

    # Si faltan, buscar en toda la imagen
    if len(seeds) < num_seeds:
        coords = np.argwhere(image == 1)
        for y, x in coords:
            if (y, x) not in seeds:
                seeds.append((y, x))
            if len(seeds) >= num_seeds:
                break

    return seeds[:num_seeds]

# --- Paso 2: aplicar flood fill desde las semillas para eliminar el fondo ---
# Crear copia de la máscara binaria
mascara_solo_lumen = mascara_limpia.copy()

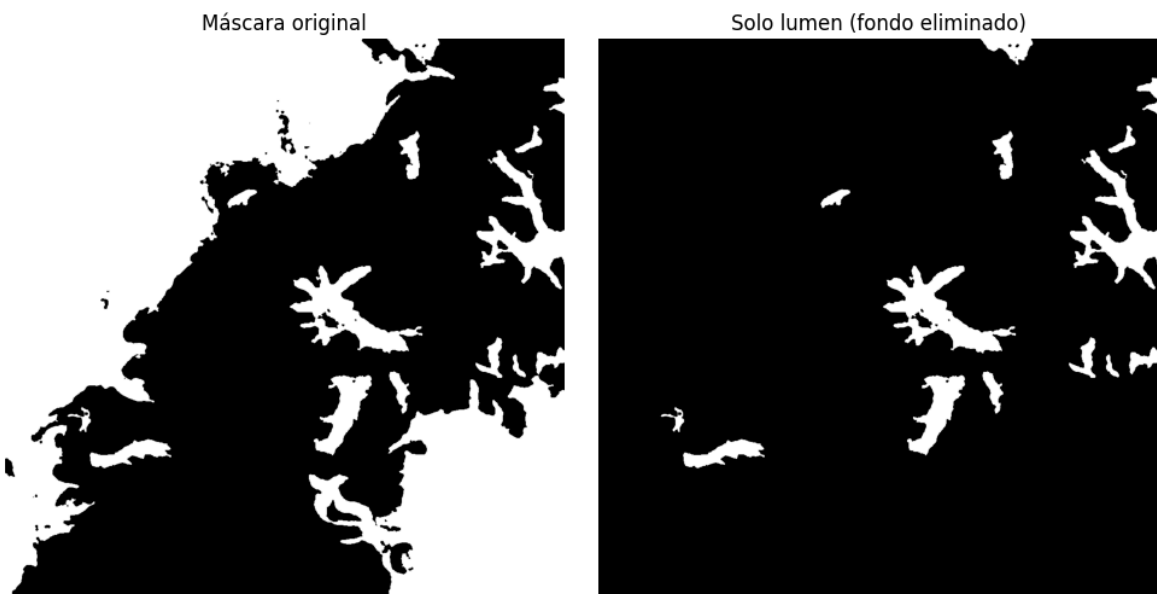
# Obtener semillas en el fondo (píxeles con valor 1)
seeds = find_seeds(mascara_limpia, num_seeds=4)

# Aplicar flood_fill para cada semilla, rellenando fondo con 0
for seed in seeds:
    mascara_solo_lumen = flood_fill(mascara_solo_lumen, seed_point=seed, new_val=0)

# Visualizar resultado
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(mascara_limpia, cmap='gray')
ax[0].set_title('Máscara original')
ax[0].axis('off')
```

```
ax[1].imshow(mascara_solo_lumen, cmap='gray')
ax[1].set_title('Solo lumen (fondo eliminado)')
ax[1].axis('off')

plt.tight_layout()
plt.show()
```



5) Rellenar los objetos de los lúmenes

Para la siguiente actividad utilizaremos la función `binary_fill_holes` de `scipy.ndimage` para rellenar posibles huecos o discontinuidades dentro de las regiones segmentadas como lúmenes.

Este paso es útil para asegurar que cada lumen esté representado como una región sólida y continua, eliminando pequeños vacíos internos que pueden generarse durante el proceso de segmentación.

El resultado es una máscara binaria más uniforme y preparada para análisis morfológicos más precisos, como el cálculo de área o forma.

```
In [6]: # Rellenar los lúmenes con la función binary_fill_holes de la librería scipy.ndi
from scipy.ndimage import binary_fill_holes

# Rellenar los lúmenes en la imagen segmentada
lumen_rellenos = binary_fill_holes(mascara_solo_lumen > 0)

# Visualizar la máscara resultante
plt.imshow(lumen_rellenos, cmap='gray')
plt.axis('off')
plt.title('Lúmenes rellenados')
plt.show()
```

Lúmenes rellenados



6) Detectar y dibujar los contornos de los lúmenes sobre la imagen original

Una vez rellenados los lúmenes detectados en la imagen, realizaremos un código que utiliza la función `find_contours` de `skimage.measure` para detectar los bordes de las regiones segmentadas como lúmenes, a partir de la máscara binaria ya rellenada (`lumen_rellenos`).

Luego, se sobreponen los contornos detectados sobre la imagen original utilizando líneas amarillas (`color='yellow'`). Esta visualización permite validar visualmente que los lúmenes han sido correctamente segmentados y localizados en su contexto anatómico original.

El uso de contornos mejora la interpretación y facilita futuras etapas de análisis morfológico o comparación entre regiones.

```
In [7]: from skimage.measure import find_contours

# Asegurarse de que la imagen original está en rango [0,1]
img = img / 255.0 if img.max() > 1.0 else img.copy()

# Detectar contornos a partir de la máscara de lúmenes rellenados
contours = find_contours(lumen_rellenos, level=0.5)

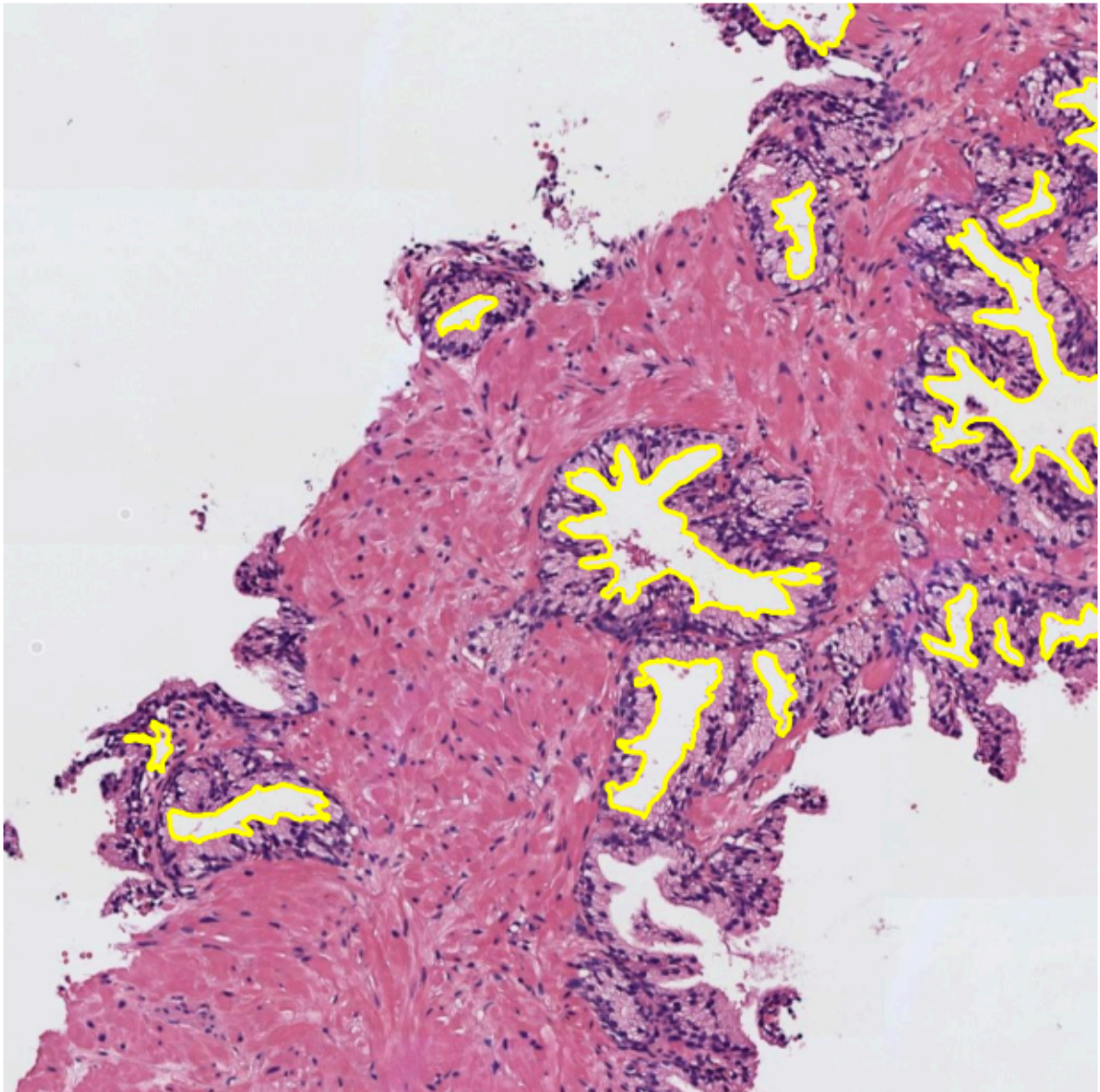
# Visualizar la imagen original y dibujar contornos en amarillo
fig, ax = plt.subplots(figsize=(8, 8))
ax.imshow(img)

# Dibujar contornos (invirtiendo coordenadas ya que vienen en (fila, columna))
for contour in contours:
```

```
ax.plot(contour[:, 1], contour[:, 0], color='yellow', linewidth=2)

ax.set_title('Lúmenes contorneados en color amarillo')
ax.axis('off')
plt.show()
```

Lúmenes contorneados en color amarillo



7) Identificar y cropear el lumen más grande

Este código localiza el **lumen de mayor tamaño** en la máscara binaria de lúmenes rellenados (`lumen_rellenos`) utilizando `label` y `regionprops` para identificar y medir cada región conectada.

Una vez identificada la región con mayor área, se extrae un recorte (crop) de la imagen original RGB correspondiente al bounding box de dicho lumen.

Esta operación permite analizar de forma individual y detallada el lumen principal de la muestra, aislándolo visualmente del resto del tejido para futuras mediciones o visualización específica.

```
In [8]: from skimage.measure import label, regionprops
```

```

# Etiquetar regiones en la máscara binaria de lúmenes rellenos
etiquetado = label(lumen_rellenos)

# Obtener propiedades de las regiones
regiones = regionprops(etiquetado)

# Seleccionar la región de mayor área (lumen más grande)
region_mayor = max(regiones, key=lambda r: r.area)

# Obtener bounding box del lumen más grande
minr, minc, maxr, maxc = region_mayor.bbox

# Cropear imagen RGB original
cropped_lumen = img[minr:maxr, minc:maxc]

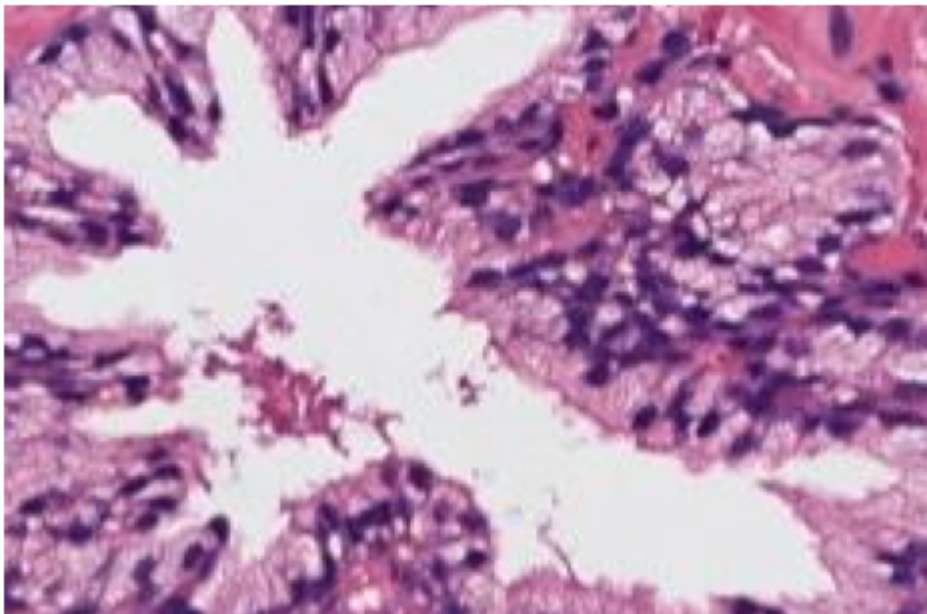
# Cropear la máscara correspondiente
cropped_mask = (etiquetado[minr:maxr, minc:maxc] == region_mayor.label)

# Recorte del lumen más grande sobre fondo original (sin alteraciones)
cropped_lumen = img[minr:maxr, minc:maxc]

# Visualizar el lumen cropeado tal cual
plt.figure(figsize=(6, 6))
plt.imshow(cropped_lumen)
plt.title('Lumen más grande')
plt.axis('off')
plt.show()

```

Lumen más grande



8) Extraer 13 características geométricas que permitan caracterizar el lumen recortado

Este código extrae una serie de propiedades geométricas y morfológicas del **lumen más grande** previamente segmentado y recortado. Utiliza funciones de `skimage.measure` y `scipy.spatial` para calcular las siguientes métricas clave:

1. **Área:** número total de píxeles que conforman el lumen.

2. **Área de la bounding box:** área del rectángulo que contiene completamente el lumen.
3. **Área convexa:** área del polígono convexo que envuelve al lumen.
4. **Excentricidad:** mide cuánto se desvía la forma de una circunferencia perfecta.
5. **Diámetro equivalente:** diámetro de un círculo con la misma área que el lumen.
6. **Extensión:** relación entre el área del lumen y el área de su bounding box.
7. **Diámetro de Feret:** distancia máxima entre dos puntos del contorno.
8. **Ejes mayor y menor:** longitudes del eje mayor y menor de la elipse ajustada al lumen.
9. **Orientación:** ángulo de inclinación del eje mayor respecto al eje horizontal.
10. **Perímetro:** longitud del borde del lumen.
11. **Solidez:** relación entre el área y el área convexa (indica irregularidad).
12. **Compacidad:** medida que relaciona perímetro y área, útil para evaluar cuán compacta es la forma.

Todo esto lo muestra en un output que podrá ser fundamental para un análisis de estructuras de este tipo. Esta salida la utilizaremos para cumplimentar los datos de nuestro trabajo.

```
In [9]: from scipy.spatial import ConvexHull, distance

# Crear máscara booleana del lumen más grande ya cropeado
# `cropped_mask` debe venir del paso anterior
props = regionprops(label(cropped_mask.astype(int)))[0]

# Obtener puntos del contorno
contour_points = np.column_stack(np.nonzero(cropped_mask)) # (y, x)

# 1) Área
area = np.round(props.area, 4)
# 2) Área de la bounding box
bbox_area = np.round((props.bbox[2] - props.bbox[0]) * (props.bbox[3] - props.bb
# 3) Área convexa
hull = ConvexHull(contour_points)
convex_area = np.round(hull.volume, 4)
# 4) Excentricidad
eccentricity = np.round(props.eccentricity, 4)
# 5) Diámetro equivalente
equiv_diameter = np.round(props.equivalent_diameter, 4)
# 6) Extensión
extent = np.round(props.extent, 4)
# 7) Diámetro Feret (máxima distancia entre dos puntos del contorno)
feret_diameter = np.round(np.max(distance.pdist(contour_points)), 4)
# 8) Longitud del eje mayor
major_axis_length = np.round(props.major_axis_length, 4)
# 9) Longitud del eje menor
minor_axis_length = np.round(props.minor_axis_length, 4)
# 10) Orientación
orientation = np.round(props.orientation, 4)
# 11) Perímetro
perimeter = np.round(props.perimeter, 4)
# 12) Solidez
solidity = np.round(props.solidity, 4)
# 13) Compacidad
```

```
compacity = np.round((perimeter**2) / (4 * np.pi * area), 4)

# Mostrar resultados
features = {
    "Área": area,
    "Área bounding box": bbox_area,
    "Área convexa": convex_area,
    "Excentricidad": eccentricity,
    "Diámetro equivalente": equiv_diameter,
    "Extensión": extent,
    "Diámetro Feret": feret_diameter,
    "Longitud eje mayor": major_axis_length,
    "Longitud eje menor": minor_axis_length,
    "Orientación": orientation,
    "Perímetro": perimeter,
    "Solidez": solidity,
    "Compacidad": compacity
}

for k, v in features.items():
    print(f"{k}: {v}")
```

Área: 13922.0
Área bounding box: 38880
Área convexa: 28485.0
Excentricidad: 0.8458
Diámetro equivalente: 133.1392
Extensión: 0.3581
Diámetro Feret: 256.3123
Longitud eje mayor: 236.4893
Longitud eje menor: 126.1803
Orientación: 1.0126
Perímetro: 1197.4194
Solidez: 0.4836
Compacidad: 8.1956